

Implementing a hybrid geolocation approach on **The Things Stack (TTS)** is a multi-step process. Because TTS is primarily a network server (it handles the *delivery* of data), it doesn't "solve" the location itself. Instead, it provides the raw metadata (RSSI, SNR, and Timestamps) to a **Location Engine**.

Here is how you can implement this specifically for a 2-gateway setup.

1. Gateway Requirements

For TDoA to work at all, your two gateways must support **Fine Timestamping**.

- **Hardware:** You need gateways with Semtech SX1302/SX1303 chips (like the RAK7289 or The Things Gateway) and a built-in GPS/GNSS.
- **Sync:** The gateways must have a "GPS Lock" to ensure their internal clocks are synchronized to within nanoseconds. Without this, TDoA calculation is impossible.

2. Enabling the "LoRa Cloud" Integration

The easiest way to implement this on TTS is through the **Semtech LoRa Cloud** integration. TTS has a native "Payload Formatter" and "Integration" for this.

1. **Get a LoRa Cloud Token:** Sign up at [LoRa Cloud](#) and get an API key for the **Modem & Geolocation Services**.
2. **Configure TTS:**
 - Go to your **Application** in the TTS Console.
 - Select **Integrations > LoRa Cloud**.
 - Enable the **Geolocation** service and paste your API key.
3. **The Process:** When a sensor sends a packet, TTS bundles the RSSI and fine-timestamps from both gateways into a JSON metadata object and sends it to LoRa Cloud. LoRa Cloud runs the hybrid TDoA/RSSI math and returns a latitude/longitude.

3. The Custom Hybrid Logic (The "Solver")

If you want to build your own solver to combine TDoA and RSSI manually (or if LoRa Cloud is too expensive), you should use a **Webhook** to send the metadata to your own server (Python/Node.js).

The Data Object

TTS will send a `rx_metadata` array in the uplink message. You need these fields:

- `gateway_ids`: To know the (x, y) of Gateway A and B.
- `rssi`: For the Proximity/Differential calculation.

- `fine_timestamp`: For the TDoA hyperbola.

The Algorithm (High Level)

To combine them in code, you would typically use a **Kalman Filter** or a **Particle Filter**:

1. **Define the Search Space**: Create a grid of possible coordinates around the gateways.
2. **TDoA Probability**: For each point in the grid, calculate what the time difference *should* be and compare it to the actual `fine_timestamp` difference. Assign a probability score.
3. **RSSI Probability**: Use the **Log-Distance Path Loss Model** to calculate the probability of the device being at that grid point based on the RSSI.

$$P(d) = P(d_0) - 10n \log_{10}\left(\frac{d}{d_0}\right)$$
4. **Multiply Probabilities**: Multiply the TDoA score by the RSSI score for every point. The point with the highest combined score is your location.

4. Limitations in The Things Stack

- **Packet Metadata**: If you are on the **Free Tier (Community Edition)**, fine-timestamping data is sometimes stripped or restricted depending on the gateway's connection type (UDP vs. Basic Station). Ensure your gateways use **LoRa Basics Station** for the most reliable timing data.
- **Gateways in View**: If one gateway is significantly further away, the `fine_timestamp` might be missing or "noisy," making the TDoA hyperbola wildly inaccurate. In that case, the system should fall back to **RSSI-only** automatically.

Summary Checklist

1. **Gateways**: Must have GPS and SX1302/1303.
2. **Protocol**: Use LoRa Basics Station.
3. **Integration**: Use the LoRa Cloud Integration in TTS for a "no-code" solution.
4. **Custom**: Use Webhooks + a Python script for a custom hybrid solver.

Do you have access to gateways that support fine-timestamping, or are you working with standard RSSI-only hardware?